

3.2. Nền tảng Trí tuệ nhân tạo

3.2.1. Tổng quan về Học máy và Dự báo chuỗi thời gian

Dự báo chuỗi thời gian là một trong những bài toán cốt lõi của Học máy (Machine Learning) và Khoa học dữ liệu, đặc biệt quan trọng trong các lĩnh vực có tính động lực học cao như tài chính, khí tượng và giao thông vận tải. Khác với các dữ liệu dạng bảng tĩnh, dữ liệu chuỗi thời gian chứa đựng chiều thông tin thứ tư – chiều thời gian, đòi hỏi các phương pháp tiếp cận chuyên biệt.

a. Khái niệm chuỗi thời gian (Time-series Data) và Đặc tính dữ liệu giao thông

Chuỗi thời gian là một tập hợp các quan trắc dữ liệu được thu thập liên tiếp theo các khoảng thời gian đều đặn (ví dụ: mỗi phút, mỗi 15 phút, mỗi giờ). Đối với hệ thống giao thông đô thị, dữ liệu cảm biến thu về (như vận tốc dòng xe, lưu lượng, độ trễ) là một chuỗi thời gian điển hình mang ba đặc trưng nền tảng:

- **Tính phụ thuộc thời gian (Temporal Dependence / Autocorrelation):** Trạng thái giao thông tại thời điểm hiện tại tại t có mối tương quan tuyến tính hoặc phi tuyến mạnh mẽ với trạng thái ở các thời điểm trước đó ($t-1$, $t-2$). Cụ thể, một đám kẹt xe không hình thành tức thời mà là sự tích lũy quán tính từ sự sụt giảm vận tốc của các chu kỳ trước.
- **Tính mùa vụ và Chu kỳ (Seasonality & Cyclicity):** Giao thông đô thị tuân theo nhịp độ sinh hoạt của con người. Dữ liệu thể hiện tính chu kỳ rõ rệt theo ngày (đỉnh điểm vào giờ cao điểm sáng/chiều) và tính mùa vụ theo tuần (mô hình di chuyển ngày thường khác biệt hoàn toàn so với cuối tuần).
- **Tính ngẫu nhiên và Hỗn loạn (Randomness & Chaos):** Bên cạnh các quy luật mang tính chu kỳ, hệ thống giao thông còn chịu sự chi phối của các biến cố ngoại lai ngẫu nhiên không thể lường trước (như tai nạn, thời tiết cực đoan, sự kiện công cộng). Yếu tố này tạo ra các điểm dị thường (Outliers), phá vỡ tính ổn định của chuỗi thời gian.

b. Mô hình thống kê truyền thống (ARIMA/SARIMA)

Trước khi kỷ nguyên Học sâu (Deep Learning) bùng nổ, các mô hình thống kê học truyền thống là công cụ tiêu chuẩn để xử lý dữ liệu chuỗi thời gian. Nổi bật nhất là mô hình **ARIMA** (AutoRegressive Integrated Moving Average) và biến thể mở rộng **SARIMA** (bao gồm yếu tố mùa vụ - Seasonal).

- **Nguyên lý về tính dừng (Stationarity):** Yêu cầu tiên quyết để các mô hình thống kê này hoạt động là dữ liệu phải có "Tính dừng". Một chuỗi thời gian được coi là dừng khi các đặc trưng thống kê cốt lõi của nó (Kỳ vọng/Trung bình μ và Phương sai σ^2) không thay đổi theo thời gian.
- **Các thành phần cấu trúc của ARIMA:** Mô hình ARIMA được ký hiệu là $ARIMA(p, d, q)$, kết hợp ba yếu tố toán học:
 1. **Thành phần Tự hồi quy - AR (AutoRegressive, bậc p):** Sử dụng kết hợp tuyến tính của p giá trị quá khứ để dự báo tương lai. Nó giả định rằng giá trị hiện tại bị ảnh hưởng trực tiếp bởi lịch sử của chính nó.

2. **Thành phần Tích hợp - I (Integrated, bậc d):** Là số lần sai phân (differencing) cần thiết để biến đổi một chuỗi thời gian không dừng thành một chuỗi dừng. Sai phân bậc 1 được tính bằng: $y'_t = y_t - y_{t-1}$.
3. **Thành phần Trung bình trượt - MA (Moving Average, bậc q):** Sử dụng các sai số dự báo trong quá khứ (lỗi ngẫu nhiên ϵ_t) để thiết lập mô hình cho giá trị hiện tại, giúp làm mịn các cú sốc đột ngột.

Phương trình toán học tổng quát của mô hình ARIMA có dạng:

$$y'_t = c + \phi_1 y'_{t-1} + \dots + \phi_p y'_{t-p} + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q} + \epsilon_t$$

(Trong đó: y'_t là chuỗi đã lấy sai phân, ϕ là hệ số tự hồi quy, θ là hệ số trung bình trượt, c là hằng số và ϵ_t là nhiễu trắng).

Hạn chế: Mặc dù sở hữu nền tảng toán học vững chắc, ARIMA/SARIMA gặp khó khăn lớn khi áp dụng vào dữ liệu giao thông thực tế. Giao thông là một hệ động lực phi tuyến phức tạp (Non-linear Dynamical System) và thường xuyên vi phạm giả định về "Tính dừng". Sự biến động dữ liệu của vận tốc dòng xe khiến các mô hình tuyến tính này phản ứng chậm và thiếu chính xác trước các sự cố vỡ trận bất ngờ.

c. Học có giám sát (Supervised Learning) trong bài toán Phân lớp đa nhãn

Để vượt qua các rào cản của mô hình tuyến tính, phương pháp Học có giám sát (Supervised Learning) thuộc nhánh Học máy/Học sâu được áp dụng. Trong bối cảnh cảnh báo sớm giao thông, thay vì cố gắng dự báo chính xác một con số liên tục (Hồi quy - Regression) như vận tốc sẽ là 23.5 km/h, bài toán được định dạng lại thành **Phân lớp đa nhãn (Multi-class Classification)**.

- **Rời rạc hóa không gian trạng thái:** Biến mục tiêu liên tục (Chỉ số kẹt xe hoặc Vận tốc) được ánh xạ (mapping) và chia thành các cấp độ phân loại rời rạc (Ví dụ: từ Lớp 0 - Thông thoáng, đến Lớp 5 - Ùn tắc nghiêm trọng).
- **Cơ chế hoạt động:** Hệ thống Học có giám sát cần một bộ dữ liệu lịch sử đã được gán nhãn (Labeled Data). Mô hình (có thể là Rừng ngẫu nhiên, Máy véc-tơ hỗ trợ SVM, hoặc Mạng Nơ-ron nhân tạo DNN) sẽ học hàm ánh xạ $f(X) \rightarrow Y$. Trong đó X là ma trận đặc trưng đầu vào (Vận tốc quá khứ, Thời gian, Tọa độ) và Y là xác suất thuộc về một trong các nhãn phân lớp kẹt xe.
- **Ưu điểm:** Cách tiếp cận phân lớp này cực kỳ phù hợp cho các hệ thống cảnh báo, vì người quản trị hệ thống và người dùng cuối quan tâm đến *mức độ nghiêm trọng của trạng thái* (để đưa ra quyết định hành động) hơn là sự xê dịch nhỏ của các con số vận tốc tuyệt đối. Thêm vào đó, các mô hình Học sâu giám sát có khả năng xấp xỉ các hàm phi tuyến cực kỳ phức tạp mà ARIMA không thể nắm bắt.

3.2.2. Mạng Nơ-ron sâu (Deep Learning)

Để xử lý khối lượng dữ liệu khổng lồ và nắm bắt các quy luật phi tuyến phức tạp mà các mô hình thống kê truyền thống không thể giải quyết, kiến trúc Mạng Nơ-ron sâu (Deep Neural Networks - DNN) đóng vai trò là hạt nhân tính toán. Mục này trình bày các lý thuyết nền tảng cấu thành nên hệ thống dự báo.

a. Mạng Nơ-ron tái phát (RNN) và Mạng bộ nhớ ngắn hạn dài (LSTM)

Mạng Nơ-ron truyền thẳng (Feed-forward Neural Network) tiêu chuẩn không có cơ chế lưu trữ trạng thái quá khứ, do đó hoàn toàn bất lực trước dữ liệu chuỗi thời gian. Mạng Nơ-ron tái phát (Recurrent Neural Network - RNN) khắc phục điều này bằng cách sử dụng các vòng lặp (loops) cho phép thông tin lưu lại trong mạng. Tuy nhiên, khi huấn luyện với các chuỗi dữ liệu dài, RNN truyền thống vấp phải vấn đề **Tiêu biến đạo hàm (Vanishing Gradient)**, khiến mạng mất khả năng học các phụ thuộc xa trong quá khứ.

Để giải quyết triệt để rào cản này, kiến trúc **Mạng bộ nhớ ngắn hạn dài (Long Short-Term Memory - LSTM)** được giới thiệu. Sức mạnh của LSTM nằm ở cấu trúc **Tế bào nhớ (Memory Cell)** phức tạp, hoạt động như một băng chuyền thông tin chạy xuyên suốt chuỗi thời gian, kết hợp với cơ chế điều tiết thông tin tinh vi thông qua 3 loại cổng (Gates):

1. **Cổng Quên (Forget Gate - f_t):** Quyết định lượng thông tin nào từ quá khứ cần bị loại bỏ khỏi tế bào nhớ. Hàm Sigmoid xuất ra giá trị từ 0 (quên hoàn toàn) đến 1 (giữ lại toàn bộ).
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
2. **Cổng Đầu vào (Input Gate - i_t):** Quyết định thông tin mới nào từ hiện tại sẽ được cập nhật vào tế bào nhớ. Nó bao gồm một tầng Sigmoid để chọn lọc và một tầng Tanh để tạo ra vector giá trị ứng viên $\tilde{C}_t$.
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
3. **Trạng thái Tế bào (Cell State - C_t):** Là sự kết hợp tuyến tính giữa trí nhớ cũ (đã lọc qua cổng quên) và thông tin mới (đã lọc qua cổng đầu vào). Đây là yếu tố cốt lõi giúp LSTM bảo toàn được đạo hàm khi truyền ngược qua nhiều bước thời gian.
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$
4. **Cổng Đầu ra (Output Gate - o_t):** Tính toán trạng thái ẩn (h_t) cho bước tiếp theo dựa trên trạng thái tế bào đã được cập nhật.
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Nhờ cơ cấu này, LSTM có khả năng "ghi nhớ" (bắt giữ phụ thuộc dài hạn - Long-term dependencies) các tín hiệu ứn tắc tích tụ từ nhiều giờ trước, đồng thời "quên đi" các nhiễu loạn cục bộ không mang ý nghĩa.

b. Tầng nhúng (Embedding Layer)

Trong dữ liệu giao thông, bên cạnh các biến liên tục (vận tốc, lưu lượng), còn tồn tại rất nhiều biến phân loại (Categorical Data) mang tính định danh ngữ cảnh, chẳng hạn như: ID đoạn đường, Ngày trong tuần, hoặc Giờ trong ngày.

Phương pháp mã hóa truyền thống phổ biến nhất là One-Hot Encoding. Tuy nhiên, nếu áp dụng One-Hot cho một mạng lưới giao thông có hàng ngàn đoạn đường, ma trận đầu vào sẽ trở nên thưa thớt (Sparse Matrix) và vấp phải hội chứng **Bùng nổ số chiều (Curse of Dimensionality)**. Điều này vắt kiệt tài nguyên tính toán bộ nhớ và làm mô hình rất khó hội tụ.

Tầng nhúng (Embedding Layer) là giải pháp kiến trúc tối ưu để thay thế. Về mặt lý thuyết, Tầng nhúng thực hiện một phép ánh xạ tuyến tính, chuyển đổi các giá trị rời rạc trong không

gian chiều cao thành các véc-tơ liên tục, trữ nặng thông tin (Dense Vectors) trong không gian chiều thấp hơn rất nhiều.

- Khác với One-Hot Encoding cố định, các trọng số của Tầng nhúng là các tham số có thể học được (Learnable Parameters) trong quá trình huấn luyện bằng Gradient Descent.
- Hệ quả là, mô hình có thể tự động khám phá và biểu diễn các mối quan hệ ngữ nghĩa (Semantic Relationships). Ví dụ: Hai đoạn đường \$A\$ và \$B\$ dù cách xa nhau về mặt địa lý nhưng nếu có cùng chung đặc tính là "thường xuyên kẹt xe vào chiều thứ 6", vector nhúng của chúng sẽ tự động xích lại gần nhau trong không gian Euclidean nhiều chiều.

c. Các hàm tối ưu và hàm mất mát nâng cao (Advanced Loss Functions)

Trong học sâu, hàm mất mát (Loss Function) định hướng cách mô hình cập nhật trọng số. Các hàm cơ bản như Cross-Entropy hoặc Mean Squared Error (MSE) thường thất bại khi đối mặt với dữ liệu mất cân bằng hoặc chứa nhiễu.

- **Focal Loss (Khắc phục mất cân bằng lớp):**
Trong bài toán phân lớp, Cross-Entropy tiêu chuẩn tính tổng lỗi bình đẳng cho mọi mẫu. Hệ quả là các lớp đa số (ví dụ: trạng thái giao thông thông thoáng) dù rất dễ dự báo nhưng với số lượng áp đảo sẽ tạo ra một dốc đạo hàm khổng lồ, nhấn chìm tín hiệu học tập từ các sự cố thảm họa hiếm gặp.
Focal Loss định hình lại khái niệm này bằng cách thêm vào một nhân tử điều biến $(1 - p_t)^\gamma$. Công thức toán học:
$$FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

(Trong đó p_t là xác suất mô hình dự báo đúng nhãn thực tế, $\gamma > 0$ là tham số tập trung).
Khi một mẫu được phân loại dễ dàng ($p_t \rightarrow 1$), nhân tử điều biến tiến về 0, tự động dập tắt trọng số của mẫu đó. Cơ chế này ép mạng Nơ-ron dồn toàn bộ sự chú ý (Attention) vào các mẫu ranh giới "khó nhận" thuộc nhóm thiểu số.
- **Huber Loss / Smooth L1 Loss (Khắc phục nhiễu ngoại lai):**
Đối với các tác vụ xấp xỉ hàm giá trị (như hồi quy hoặc ước lượng phần thưởng trong Học tăng cường), việc sử dụng MSE (L_2 Loss) rất nhạy cảm với dữ liệu ngoại lai (Outliers) - vốn là đặc sản của dữ liệu cảm biến IoT. Một điểm nhiễu duy nhất bị bình phương lên có thể làm nổ gradient (Exploding Gradients). Ngược lại, MAE (L_1 Loss) lại có đạo hàm không liên tục tại điểm 0, gây khó khăn khi hội tụ.
Huber Loss là sự kết hợp hoàn hảo giữa hai hàm trên:
$$L_\Delta(a) = \begin{cases} \frac{1}{2} a^2 & \text{với } |a| \leq \Delta \\ |a| - \frac{1}{2} \Delta & \text{trường hợp khác} \end{cases}$$

(Trong đó a là sai số dự báo, Δ là ngưỡng chuyển đổi).
Đối với các sai số nhỏ ($|a| \leq \Delta$), hàm hoạt động như MSE, giúp mô hình hội tụ mượt mà quanh điểm cực tiểu. Đối với các sai số lớn do nhiễu, hàm chuyển sang dạng tuyến tính của MAE, bảo vệ mạng Nơ-ron khỏi sự phá hoại của các tín hiệu vật lý vô lý.

3.2.3. Học tăng cường sâu (Deep Reinforcement Learning - DRL)

Học tăng cường sâu là sự kết hợp đột phá giữa khả năng cảm nhận của Học sâu (Deep Learning) và khả năng ra quyết định của Học tăng cường (Reinforcement Learning). Trong bài toán dự báo giao thông, DRL không chỉ học cách "nhận diện" kẹt xe mà còn học được "chiến lược" dự báo để tối ưu hóa lợi ích lâu dài cho toàn hệ thống.

a. Quy trình quyết định Markov (Markov Decision Process - MDP)

Mọi bài toán Học tăng cường đều được mô hình hóa dưới dạng một Quy trình quyết định Markov. Đây là một khung toán học để mô tả việc ra quyết định trong các tình huống mà kết quả vừa mang tính ngẫu nhiên, vừa chịu ảnh hưởng của người ra quyết định. MDP được định nghĩa bởi một bộ 5 thành phần (S, A, P, R, γ) :

1. **Đặc vụ (Agent):** Là "não bộ" của hệ thống (mô hình dự báo), thực hiện việc quan sát và đưa ra hành động.
2. **Môi trường (Environment):** Thế giới thực tế (mạng lưới giao thông) nơi Đặc vụ hoạt động.
3. **Trạng thái (State - S):** Tập hợp các quan sát tại thời điểm t (vận tốc, mật độ, lịch sử di chuyển).
4. **Hành động (Action - A):** Tập hợp các quyết định mà Đặc vụ có thể thực hiện (dự báo mức độ từ 0 đến 5).
5. **Phần thưởng (Reward - R):** Tín hiệu phản hồi từ môi trường sau mỗi hành động (điểm cộng nếu dự báo đúng, điểm trừ nếu dự báo sai lệch).
6. **Hệ số chiết khấu (Discount Factor - γ):** Xác định tầm quan trọng của các phần thưởng trong tương lai so với phần thưởng tức thì.

b. Thuật toán Q-Learning và Deep Q-Network (DQN)

Trong Q-Learning truyền thống, Đặc vụ sử dụng một bảng tra cứu (Q-table) để lưu trữ giá trị $Q(s, a)$ - đại diện cho "chất lượng" của việc thực hiện hành động a khi đang ở trạng thái s . Tuy nhiên, với không gian trạng thái giao thông khổng lồ và liên tục, việc sử dụng bảng là bất khả thi.

Deep Q-Network (DQN) giải quyết vấn đề này bằng cách sử dụng một **Mạng Nơ-ron sâu** để xấp xỉ hàm giá trị Q . Thay vì tra bảng, Đặc vụ đưa trạng thái s vào mạng nơ-ron và nhận về các giá trị Q cho tất cả các hành động có thể. Mục tiêu của mạng là cực tiểu hóa sai số Bellman:

$$J(\theta) = \mathbb{E} \left[\left(R + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta) \right)^2 \right]$$

c. Thuật toán Double DQN (DDQN) và việc triệt tiêu ảo tưởng giá trị

Một hạn chế lớn của DQN truyền thống là **Hội chứng ảo tưởng giá trị (Overestimation Bias)**. Do toán tử $\max$ trong phương trình Bellman luôn chọn giá trị lớn nhất, mạng nơ-ron có xu hướng đánh giá quá cao các hành động có nhiều dương, dẫn đến việc Agent đưa ra các quyết định sai lầm nhưng lại "tự tin" là đúng.

Double DQN (DDQN) khắc phục điều này bằng cách tách biệt việc **Lựa chọn hành động** và **Đánh giá hành động** thông qua hai mạng:

- **Mạng Chính (Policy Network - θ):** Dùng để chọn hành động có giá trị Q cao nhất.
- **Mạng Đích (Target Network - θ^-):** Dùng để tính toán giá trị của hành động đã chọn đó.

Công thức mục tiêu của DDQN trở thành:

$$Y_t^{DDQN} = R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t); \theta^-_t)$$

Cơ chế này đảm bảo Đặc vụ nhìn nhận môi trường một cách khách quan hơn, đặc biệt quan trọng trong việc phát hiện các trạng thái "thảm họa" (Lớp 5) vốn thường bị nhiễu bởi các lớp kẹt xe nhẹ.

d. Cơ chế Bộ nhớ kinh nghiệm (Experience Replay)

Trong giao thông, các trạng thái liên tiếp (giây thứ nhất và giây thứ hai) thường rất giống nhau, tạo ra tính **tương quan chuỗi (Temporal Correlation)** cực cao. Nếu mạng nơ-ron chỉ học trực tiếp từ dữ liệu luồng này, nó sẽ dễ dàng bị chệch hướng và không thể hội tụ.

Replay Buffer (Bộ nhớ kinh nghiệm) hoạt động như một kho lưu trữ các trải nghiệm quá khứ dưới dạng bộ tứ (s, a, r, s') .

- Trong quá trình huấn luyện, Đặc vụ không học từ dữ liệu mới nhất mà lấy ra một "Batch" ngẫu nhiên từ bộ nhớ này.
- Việc xáo trộn dữ liệu (Random Sampling) giúp phá vỡ tính tương quan thời gian, đảm bảo dữ liệu huấn luyện mang tính độc lập và phân phối đồng nhất (i.i.d), giúp quá trình hội tụ ổn định và bền bỉ hơn.

3.2.4. Các kỹ thuật xử lý dữ liệu và mô hình sinh tạo

Để xây dựng một hệ thống Trí tuệ Nhân tạo hoạt động hiệu quả trong môi trường giao thông thực tế, việc thiết kế mạng Nơ-ron (như LSTM hay DDQN) chỉ giải quyết được một nửa bài toán. Nửa còn lại, và thường là phần khó khăn nhất, nằm ở việc xử lý các rào cản và khiếm khuyết của dữ liệu đầu vào.

a. Vấn đề mất cân bằng dữ liệu (Data Imbalance) và Nghịch lý độ chính xác

Dữ liệu giao thông đô thị là một ví dụ kinh điển về phân phối mất cân bằng cực hạn (Extreme Class Imbalance). Trong một tập dữ liệu chuẩn, trạng thái giao thông bình thường (Lớp 0, 1) thường chiếm tới 90% - 95% tổng thời lượng, trong khi các sự cố ùn tắc nghiêm trọng hoặc vỡ trận (Lớp 4, 5) chỉ chiếm một tỷ lệ rất nhỏ ($< 5\%$).

- **Nghịch lý độ chính xác (Accuracy Paradox):** Khi huấn luyện mô hình học máy trên dữ liệu này, nếu sử dụng chỉ số **Độ chính xác tổng thể (Accuracy)** làm mục tiêu tối ưu, mô hình sẽ dễ dàng rơi vào một cái bẫy thống kê. Cụ thể, một mô hình "lười biếng" (ZeroR) nếu luôn luôn dự báo trạng thái là "Thông thoáng" bất chấp dữ liệu đầu vào, nó vẫn ngẫu nhiên đạt Accuracy 95%. Tuy nhiên, giá trị thực tiễn của nó bằng 0\$ vì nó bỏ lỡ 100% các thảm họa cần cảnh báo.
- **Tầm quan trọng của F1-Score và Recall:** Để đánh giá chính xác năng lực phát hiện thảm họa, các chỉ số sau được ưu tiên sử dụng:

- **Độ phủ (Recall / Sensitivity):** Tỷ lệ phần trăm các vụ kẹt xe thực tế được hệ thống phát hiện thành công ($\frac{TP}{TP + FN}$). Trong hệ thống cảnh báo sớm, tối đa hóa Recall (giảm thiểu bỏ lọt) là ưu tiên sống còn.
- **F1-Score:** Là trung bình điều hòa giữa Precision (Độ chuẩn xác) và Recall, giúp đánh giá một cách cân bằng khi mô hình không thể hy sinh hoàn toàn Precision chỉ để lấy Recall.

b. Mạng đối nghịch sinh tạo (Generative Adversarial Networks - GAN)

Để giải quyết tận gốc vấn đề mất cân bằng, thay vì nhân bản dữ liệu cũ (Over-sampling truyền thống như SMOTE) dễ gây ra tình trạng học vẹt (Overfitting), công nghệ Trí tuệ Nhân tạo sinh được áp dụng thông qua **Mạng đối nghịch sinh tạo (GAN)**.

Khung thuật toán GAN được Ian Goodfellow giới thiệu vào năm 2014, hoạt động dựa trên nguyên lý của **Lý thuyết trò chơi (Game Theory)**, cụ thể là một trò chơi Minimax có tổng bằng không (Zero-sum game) giữa hai mạng Nơ-ron độc lập:

1. **Mạng Sinh (Generator - \$G\$):** Đóng vai trò là "Kẻ làm giả". Nó nhận đầu vào là một vector nhiễu ngẫu nhiên z và cố gắng sinh ra các mẫu dữ liệu giả $G(z)$ sao cho phân phối của chúng tiệm cận với phân phối của dữ liệu thật.
2. **Mạng Phân biệt (Discriminator - \$D\$):** Đóng vai trò là "Cảnh sát". Nó nhận đầu vào là cả dữ liệu thật x và dữ liệu giả $G(z)$, sau đó xuất ra xác suất để phân biệt đâu là thật, đâu là giả.

Quá trình huấn luyện là một cuộc chạy đua vũ trang liên tục. Hàm mục tiêu tối ưu của GAN được biểu diễn qua phương trình:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Khi trò chơi đạt đến trạng thái cân bằng Nash (Nash Equilibrium), mạng Generator đã học được hoàn hảo các quy luật vật lý tiềm ẩn của dữ liệu thật, khiến mạng Discriminator không thể phân biệt được nữa (xác suất đoán đúng chỉ còn 50%).

c. Conditional Tabular GAN (CTGAN) và Sinh chuỗi thời gian (Sequence Generation)

Mạng GAN nguyên thủy được thiết kế tối ưu cho dữ liệu hình ảnh (ma trận điểm ảnh liên tục). Tuy nhiên, dữ liệu giao thông lại là **dữ liệu dạng bảng (Tabular Data)** chứa cả biến liên tục (vận tốc) và biến phân loại rời rạc (ID ngã tư, mã thời tiết).

- **CTGAN (Conditional Tabular GAN):** Là một biến thể kiến trúc được tinh chỉnh riêng cho dữ liệu bảng. Nó vượt qua giới hạn của GAN truyền thống bằng cách sử dụng cơ chế Huấn luyện có Điều kiện (Conditional Generator). Thay vì sinh ngẫu nhiên, CTGAN cho phép ép buộc mô hình chỉ sinh ra dữ liệu của một lớp cụ thể (ví dụ: chỉ sinh các mẫu thuộc Lớp kẹt xe 5).
- **Tính nhất quán của Chuỗi thời gian (Temporal Consistency):** Khi áp dụng CTGAN vào hệ thống dự báo, một thách thức lớn phát sinh: Giao thông là một chuỗi thời gian liên tục. Nếu CTGAN sinh ra từng dòng dữ liệu một cách độc lập, nó sẽ vi phạm động lực học vật lý (ví dụ: vận tốc đang 60 km/h đột ngột rớt xuống

0\text{ km/h} trong một giây). Do đó, kỹ thuật **Sequence Generation** được tích hợp. CTGAN được cấu hình để sinh dữ liệu theo đơn vị **Khối cửa sổ (Windows)** (ví dụ: sinh một lượt 13 bước thời gian liên tiếp). Điều này ép mạng Discriminator phải đánh giá sự hợp lý của cả một "chuỗi diễn biến", từ đó đảm bảo dữ liệu nhân tạo bảo toàn được tính mượt mà của sóng xung kích ừn tắc.

3.2.5. Trí tuệ nhân tạo có thể giải thích (Explainable AI - XAI)

Sự phát triển bùng nổ của các mô hình Học sâu (Deep Learning) mang lại độ chính xác vượt trội, nhưng đồng thời cũng tạo ra một rào cản lớn: Tính "hộp đen" (Black-box). Mạng Nơ-ron với hàng triệu tham số phi tuyến tính không thể cung cấp cho con người một lý do trực quan để hiểu tại sao nó lại đưa ra một dự báo cụ thể. Trong các hệ thống quan trọng như y tế hay cảnh báo giao thông, sự thiếu minh bạch này dẫn đến rủi ro về độ tin cậy. Trí tuệ nhân tạo có thể giải thích (Explainable AI - XAI) ra đời nhằm mục đích bóc tách hộp đen này, giúp con người hiểu, tin tưởng và kiểm toán các quyết định của máy móc.

a. Lý thuyết giá trị Shapley (Shapley Value Theory)

Nền tảng toán học mạnh mẽ nhất của XAI hiện đại bắt nguồn từ Lý thuyết trò chơi hợp tác (Cooperative Game Theory), cụ thể là khái niệm **Giá trị Shapley**, được nhà toán học đoạt giải Nobel Lloyd Shapley giới thiệu vào năm 1953.

Lý thuyết này đặt ra một bài toán kinh điển: Giả sử một nhóm người chơi (liên minh) cùng hợp tác làm việc để đạt được một phần thưởng tổng (payout). Làm thế nào để phân chia phần thưởng này một cách công bằng nhất, tương xứng với mức độ đóng góp của từng cá nhân?

Giá trị Shapley giải quyết vấn đề này thông qua khái niệm **Đóng góp biên (Marginal Contribution)**. Đóng góp biên của một người chơi là giá trị tăng thêm khi người đó gia nhập vào một liên minh đã có sẵn. Tuy nhiên, đóng góp của một người chơi phụ thuộc rất lớn vào việc họ tham gia liên minh trước hay sau những người khác. Do đó, Giá trị Shapley của một người chơi được tính bằng trung bình cộng của tất cả các đóng góp biên trong mọi hoán vị gia nhập có thể xảy ra.

Công thức toán học nguyên thủy của Giá trị Shapley cho người chơi i là:

$$\phi_i = \sum_{S \subseteq N, i \in S} \frac{|S|! (N - |S|)!}{N!} [v(S \cup \{i\}) - v(S)]$$

Trong đó:

- N : Tập hợp tổng số người chơi (đặc trưng).
- S : Một liên minh (tập con) các người chơi không chứa i .
- $v(S)$: Giá trị (phần thưởng) mà liên minh S đạt được.
- $v(S \cup \{i\}) - v(S)$: Đóng góp biên của người chơi i khi gia nhập liên minh S .
- $\frac{|S|! (N - |S|)!}{N!}$: Trọng số hoán vị, đại diện cho xác suất liên minh S được hình thành trước khi i tham gia.

b. Phương pháp SHAP (SHapley Additive exPlanations)

Vào năm 2017, Lundberg và Lee đã công bố phương pháp **SHAP**, một bước đột phá mang Lý thuyết Shapley ứng dụng trực tiếp vào Học máy. Trong SHAP:

- "Trò chơi" chính là bài toán dự báo của mô hình.
- "Người chơi" là các đặc trưng đầu vào (Ví dụ: Vận tốc, Chỉ số kẹt xe, Thời tiết).
- "Phần thưởng tổng" là sự chênh lệch giữa giá trị dự báo cho một điểm dữ liệu cụ thể (Local Prediction) và Giá trị kỳ vọng cơ sở (Base Value / Expected Value) của toàn bộ tập dữ liệu.

Phương pháp SHAP định nghĩa một mô hình giải thích tuyến tính mang tính cộng gộp (Additive Feature Attribution Method):

$$g(z') = \phi_0 + \sum_{i=1}^M \phi_i z'_i$$

Trong đó $g(z')$ là mô hình giải thích, ϕ_0 là giá trị cơ sở (kết quả dự báo trung bình của hệ thống khi không có thông tin đặc trưng nào), và ϕ_i là giá trị SHAP (mức độ đóng góp) của đặc trưng thứ i . Nếu $z'_i = 1$ (đặc trưng có mặt), nó sẽ đóng góp ϕ_i vào tổng kết quả dự báo.

Sức mạnh diễn giải của SHAP thể hiện ở hai cấp độ:

1. **Tính diễn giải Cục bộ (Local Interpretability):** SHAP cho phép giải thích từng dự báo đơn lẻ thông qua Biểu đồ thác nước (Waterfall Plot) hoặc Biểu đồ lực (Force Plot). Nó chỉ rõ với một vụ kẹt xe vừa xảy ra, đặc trưng "Vận tốc giảm" đã đẩy xác suất kẹt xe lên bao nhiêu phần trăm, và đặc trưng "Trời quang mây tạnh" đã kéo xác suất đó xuống bao nhiêu phần trăm.
2. **Tính diễn giải Toàn cục (Global Interpretability):** Bằng cách tổng hợp giá trị SHAP tuyệt đối của tất cả các điểm dữ liệu, phương pháp này xếp hạng được Tầm quan trọng của Đặc trưng (Feature Importance) trên toàn hệ thống một cách khách quan, triệt tiêu được các sai lệch do tính tương quan chéo giữa các biến gây ra.

3.2.6. Công nghệ và Công cụ sử dụng

Để hiện thực hóa các mô hình toán học phức tạp như Mạng Nơ-ron Đồ thị, Học tăng cường (RL) hay Mạng đối nghịch sinh tạo (GAN) thành một hệ thống cảnh báo giao thông hoạt động trơn tru trong thực tế, đồ án thiết lập một hệ sinh thái công nghệ (Technology Stack) toàn diện. Hệ sinh thái này trải dài từ khâu thao tác dữ liệu thô, huấn luyện mô hình cho đến khâu tối ưu hóa triển khai (MLOps).

a. Ngôn ngữ lập trình và Thư viện lõi (Core Ecosystem)

- **Python:** Được lựa chọn làm ngôn ngữ lập trình chủ đạo của toàn bộ hệ thống. Với đặc tính cú pháp tường minh và sự hậu thuẫn từ cộng đồng khoa học dữ liệu khổng lồ, Python cung cấp một nền tảng liền mạch để tích hợp từ các tập lệnh thu thập dữ liệu IoT đến các mô hình Học sâu phức tạp.

- **PyTorch:** Đóng vai trò là "động cơ" tính toán (Compute Engine) cốt lõi cho các kiến trúc Học sâu và Học tăng cường (Double DQN). Khác với các framework sử dụng đồ thị tĩnh, sự ưu việt của PyTorch nằm ở cơ chế **Đồ thị tính toán động (Dynamic Computation Graph / Define-by-Run)**. Cơ chế này đặc biệt quan trọng khi xây dựng Môi trường Học tăng cường (RL Environment), cho phép hệ thống linh hoạt tính toán đạo hàm, cập nhật trọng số và tùy chỉnh hàm phần thưởng không đối xứng (Asymmetric Reward) ngay trong quá trình Đặc vụ tương tác với dữ liệu chuỗi thời gian.
- **Scikit-learn:** Thư viện chuẩn mực (Industry-standard) cho các tác vụ tiền xử lý và Học máy cơ bản. Trong đồ án, thư viện này đảm nhiệm vai trò thiết lập các "Bản hợp đồng dữ liệu" (Data Artifacts) như **StandardScaler** để chuẩn hóa phân phối vận tốc/độ trễ, **LabelEncoder** để mã hóa ngữ cảnh không gian, đồng thời cung cấp các thuật toán truyền thống để xây dựng mô hình đối chứng (Baseline Models).

b. Công cụ xử lý và Sinh tạo dữ liệu (Data & Generative AI Tools)

- **Pandas và NumPy:** Là xương sống của luồng dữ liệu (Data Pipeline). NumPy cung cấp cấu trúc mảng đa chiều (nd-array) với tốc độ tính toán vector hóa (Vectorization) tối ưu nhờ phần lõi được viết bằng ngôn ngữ C. Trong khi đó, Pandas xử lý xuất sắc các tác vụ tính toán chuỗi thời gian (Time-series manipulation), bao gồm kỹ thuật trượt cửa sổ thời gian (Rolling Windows) và điền khuyết tín hiệu cảm biến lỗi.
- **SDV (Synthetic Data Vault) - CTGAN:** Để giải quyết bài toán mất cân bằng dữ liệu cực hạn, thư viện SDV được tích hợp để khai thác sức mạnh của mô hình Conditional Tabular GAN. Thay vì phải tự xây dựng mạng GAN từ đầu để gặp hiện tượng sụp đổ mode (Mode Collapse), thư viện SDV cung cấp một bộ khung tối ưu chuyên dụng cho dữ liệu dạng bảng, giúp hệ thống sinh ra hàng vạn kịch bản kẹt xe nhân tạo mà vẫn tuân thủ nghiêm ngặt các ràng buộc vật lý.

c. Công nghệ Tối ưu hóa và Triển khai - MLOps (Deployment Pipeline) Bước chuyển tiếp từ mô hình trong phòng thí nghiệm (Research) sang hệ thống thực tế (Production) đòi hỏi các công nghệ tăng tốc suy luận và đóng gói tiêu chuẩn:

- **ONNX (Open Neural Network Exchange):** Mạng Q-Network sau khi huấn luyện trên PyTorch sẽ được xuất sang định dạng ONNX. Đây là một định dạng trung gian độc lập với framework, giúp hợp nhất các toán tử mạng Nơ-ron (Operator Fusion) và tinh gọn đồ thị tính toán, loại bỏ những phép toán dư thừa sinh ra trong quá trình huấn luyện.
- **TensorRT / ONNX Runtime:** Để đáp ứng độ trễ suy luận (Inference Latency) ở cấp độ phần nghìn giây khi hệ thống phải quét qua hàng vạn đoạn đường cùng lúc, ONNX Runtime và NVIDIA TensorRT được sử dụng làm Execution Engine. Các công cụ này biên dịch mô hình trực tiếp thành mã máy cấp thấp, khai thác triệt để sức mạnh xử lý song song và lượng tử hóa (Quantization) của GPU.
- **Docker:** Toàn bộ hệ sinh thái từ môi trường Python, các tệp Artifacts chuẩn hóa, cho đến lõi suy luận TensorRT đều được container hóa (Containerization) bằng Docker. Công nghệ này đảm bảo tính "Bất biến môi trường" (Environment Immutability), giải quyết triệt để rào cản "Chạy được trên máy tôi nhưng lỗi trên server" (Works on my machine), tạo tiền đề vững chắc cho kiến trúc Microservices và dễ dàng nhân bản (Scale) tải trọng theo thời gian thực.

